

Answers to the mid-term exam on Compilers

Christian Rinderknecht

25 October 2005

Question 1. Let the alphabet $\Sigma = \{a, b\}$ and the following regular expressions:

$$r = a(a|b)^*ba$$

$$s = (ab)^*|(ba)^*|(a^*|b^*)$$

The language denoted by r is noted $L(r)$ and the language denoted by s is noted $L(s)$.

Find a word x such as

1. $x \in L(r)$ and $x \notin L(s)$
2. $x \notin L(r)$ and $x \in L(s)$
3. $x \in L(r)$ and $x \in L(s)$
4. $x \notin L(r)$ and $x \notin L(s)$

Answer 1. The method to answer these questions is simply to try small words by constructing them in order to satisfy the constraints.

1. The shortest word x belonging to $L(r)$ is found by taking ε in place of $(a|b)^*$. So $x = aba$. Let us check if $x \in L(s)$ or not. $L(s)$ is made of the union of four sub-languages (subsets). To make this clear, let us remove the useless parentheses on the right side:

$$s = (ab)^*|(ba)^*|a^*|b^*$$

Therefore, membership tests on $L(s)$ have to be split into four: one membership test on $(ab)^*$, one on $(ba)^*$, one on a^* and another one on b^* . In other words:

$$x \in L(s) \Leftrightarrow x \in L((ab)^*) \text{ or } x \in L((ba)^*) \text{ or } x \in L(a^*) \text{ or } x \in L(b^*)$$

Let us test the membership with $x = aba$:

- (a) The words in $L((ab)^*)$ are $\varepsilon, ab, abab \dots$. Thus $aba \notin L((ab)^*)$.

- (b) The words in $L((ba)^*)$ are $\varepsilon, ba, baba \dots$. Hence $aba \notin L((ba)^*)$.
- (c) The words in $L(a^*)$ are $\varepsilon, a, aa \dots$. Therefore $aba \notin L(a^*)$.
- (d) The words in $L(b^*)$ are $\varepsilon, b, bb \dots$. So $aba \notin L(b^*)$.

Finally the conclusion is $aba \notin L(s)$, which is what we were looking for.

2. What is the shortest word belonging to $L(s)$? Since the four sub-languages composing $L(s)$ are starred, it means that $\varepsilon \in L(s)$. Since we showed at the item (1) that aba is the shortest word of $L(r)$, it means that $\varepsilon \notin L(r)$ because ε is of length 0.
3. This question is a bit more difficult. After a few tries, we cannot find any x such as $x \in L(r)$ and $x \in L(s)$. Then we may try to prove that $L(r) \cap L(s) = \emptyset$, i.e. there is no such x . How should we proceed? The idea is to use the decomposition of $L(s)$ into four sub-languages and try to prove

$$L(r) \cap L((ab)^*) = \emptyset$$

$$L(r) \cap L((ba)^*) = \emptyset$$

$$L(r) \cap L(a^*) = \emptyset$$

$$L(r) \cap L(b^*) = \emptyset$$

Indeed, if all these four equations are true, they imply $L(r) \cap L(s) = \emptyset$.

- (a) Any word in $L(r)$ finishes with a whereas any word in $L((ab)^*)$ finishes with b or is ε . Thus $L(r) \cap L((ab)^*) = \emptyset$.
- (b) For the same reason, $L(r) \cap L(b^*) = \emptyset$.
- (c) Any word in $L(r)$ contains both a and b whereas any word in $L(a^*)$ contains only a or is ε . Therefore $L(r) \cap L(a^*) = \emptyset$.
- (d) Any word in $L(r)$ starts with a whereas any word in $L((ba)^*)$ starts with b or is ε . Thus $L(r) \cap L((ba)^*) = \emptyset$.

Finally, since all the four equations are false, they imply that

$$L(r) \cap L(s) = \emptyset.$$

4. Let us construct letter by letter a word x which does not belong neither to $L(r)$ nor $L(s)$. First, we note that all words in $L(r)$ start with a , so we can try to start x with b : this way $x \notin L(r)$. So we have $x = b \dots$ and we have to fill the dots with some letters in such a way that $x \notin L(s)$.

We use again the decomposition of $L(s)$ into four sub-languages and make sure that x does not belong to any of those sub-languages.

First, because x starts with a , $x \notin L(b^*)$ and $x \notin L((ba)^*)$.

Now, we have to add some more letters such as $x \notin L(a^*)$ and $x \notin L((ab)^*)$.

Since any word in $L(a^*)$ has a a as second letter or is ε , we can choose the second letter of x to be b . This way $x = ab \dots \notin L(a^*)$.

Finally, we have to add more letters to make sure that

$$x = ab \dots \notin L((ab)^*)$$

Any word in $L((ab)^*)$ is either ε or ab or $abab \dots$, hence the third letter is a . Therefore, let us choose b as the third letter of x and we thus have $x = aba \notin L((ab)^*)$. Summary:

$$aba \notin L(r) \quad aba \notin L(b^*) \quad aba \notin L((ba)^*) \quad aba \notin L(a^*) \quad aba \notin L((ab)^*)$$

which is equivalent to

$$aba \notin L(r) \text{ and } aba \notin L((ab)^*) \cup L((ba)^*) \cup L(a^*) \cup L(b^*) = L(s)$$

Therefore $x = aba$ is one possible answer.

Question 2. Given the binary alphabet $\Sigma = \{a, b, c\}$ and the order on letters $a < b < c$, write regular definitions for the following languages.

1. All words starting and ending with c .
2. All words in which the third last letter is b .
3. All words containing exactly three a .
4. All words containing at least one b before a c .
5. All words in which the letters are in increasing order.

Answer 2.

1. The constraint on the words is that they must be of the shape $c \dots c$ where the dots stand for “any combination of a , b and c .” In other words, one answer is $c(a|b|c)^*c|c$.
2. The question implies that the words we are looking for are of the form $\dots b _ _$ where the dots stand for “any sequence of a , b and c ” and each $_$ stands for a regular expression denoting any letter. Any letter is described by $(a|b|c)$; therefore one possible answer is

$$(a|b|c)^*b(a|b|c)(a|b|c)$$

3. The words we search contain, at any place, exactly three a , so are of the form $\dots a \dots a \dots a \dots$, where the dots stand for “any letter except a ”, i.e., “any number of b or c .” In other words:

$$(b|c)^*a(b|c)^*a(b|c)^*a(b|c)^*$$

4. The words we search are of the form $\dots b \dots c \dots$, where the dots stand for "All words possible words made of a , b or c ." Therefore it is easy to understand that a short answer is

$$(a|b|c)^* b (a|b|c)^* c (a|b|c)^*$$

5. Because the alphabet is made only of two letters, the answer is easy: we put first all the a , then all the b and finally all the c :

$$a^* b^* c^*$$

Question 3. Simplify, if possible, the following regular expressions.

$$\begin{aligned} & (\epsilon | a^* | b^* | a | b)^* \\ & a(a|b)^* b | (ab)^* | (ba)^* \end{aligned}$$

Answer 3.

1. The first regular expression can be simplified in the following way:

$$\begin{aligned} (\epsilon | a^* | b^* | a | b)^* &= (\epsilon | a^* | b^* | b)^* && \text{since } L(a) \subset L(b^*) \\ &= (\epsilon | a^* | b^*)^* && \text{since } L(b) \subset L(b^*) \\ &= (\epsilon | a^+ | b^+)^* && \text{since } \{\epsilon\} \subset L(b^*) \\ &= (a^+ | b^+)^* && \text{since } (\epsilon | x)^* = x^* \end{aligned}$$

Words in $L((a^+ | b^+)^*)$ are of the form $(a \dots a)(b \dots b)(a \dots a)(b \dots b) \dots$, so we recognise $(a|b)^*$. Therefore $(\epsilon | a^* | b^* | a | b)^* = (a|b)^*$.

2. The second regular expression can be simplified in the following way. We note first that the expression is made of the disjunction of three regular sub-expressions (i.e. it is a union of three sub-languages). The simplest idea is then to check whether one of these sub-languages is redundant, i.e. if one is included in another. If so, we can simply remove it from the expression.

$$\begin{aligned} a(a|b)^* b | (ab)^* | (ba)^* &= a(a|b)^* b | \epsilon | (ab)^+ | (ba)^* && \text{since } (ab)^* = \epsilon | (ab)^+ \\ &= a(a|b)^* b | (ab)^+ | (ba)^* && \text{since } \{\epsilon\} \subset L((ab)^+) \end{aligned}$$

We have:

$$\begin{aligned} (ab)^+ &= (ab)(ab) \dots (ab) \\ &= a(ba)(ba) \dots (ba)b | ab \\ &= a(ba)^* b \end{aligned}$$

Also $(ba) \subset (a|b)^*$ and then $(ba)^* \subset (a|b)^*$, because $(a|b)^*$ contains all the words. Therefore $a(ba)^* b \subset a(a|b)^* b$, i.e. $(ab)^+ \subset a(a|b)^* b$.

As a consequence, one possible answer is

$$a(a|b)^*b|(ab)^*(ba)^* = a(a|b)^*b|(ba)^*$$

The intersection between $L(a(a|b)^*b)$ and $L((ba)^*)$ is empty because all the words of the former start with a , while all the words of the other start with b (or is ε). Therefore we cannot simply further this way.